Diffusion Policies for Complex Continuous Control

Kendrick Slamat
School of Computing, Singapore Polytechnic
ST1504 Deep Learning, AY2026/27 Semester 1
kendrick.25@ichat.sp.edu.sg

Abstract

Reinforcement learning agents for continuous control must select precise, coordinated actions while responding to complex environment dynamics. Deep Q-Networks (DQN) operate on discrete actions, so applying them to multidimensional continuous control requires joint-action discretisation that grows rapidly with the number of action dimensions. This study investigates whether a diffusion policy can imitate continuous expert behaviour in LunarLanderContinuous-v3 more effectively than standard behaviour cloning and a discretised DQN baseline. The comparison evaluates episodic return, successful-landing rate, action smoothness, and inference cost under repeated evaluation seeds.

1 Introduction

Reinforcement Learning (RL) is a paradigm of machine learning that focuses on training agents to make decisions by interacting with an environment. In continuous control problems, agents learn policies that produce actions within an environment that attempt to maximize overall reward over time (Milvus, n.d.; Terven, 2025), which can be written as follows:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right], \tag{1}$$

where r_t is the reward received at timestep t and $\gamma \in [0, 1)$ is the discount factor that controls how much the agent cares about rewards later on, versus rewards right now.

One approach to continuous control is to adapt the Deep Q-Network (DQN), which combines Q-learning with a neural network that estimates the value of each available action. DQN needs a finite action set, though, so each continuous action dimension has to be split into bins. For LunarLanderContinuous-v3, discretising both control dimensions independently into K bins gives K^2 joint actions — more bins buy finer control at the cost of a much larger output space. Standard deterministic policies have the same problem in a different form: they produce one preferred action per state, and can end up averaging across distinct valid behaviours in the demonstration data.

To address these limitations, this paper explores the use of diffusion models in reinforcement learning for continuous control tasks. This leads to the following research question:

> *To what extent can a diffusion policy improve imitation learning for complex continuous control in LunarLanderContinuous-v3, compared with standard behaviour cloning and a discretised DQN baseline?*

2 Background: Denoising Diffusion Models

Diffusion models are a class of generative models that can learn to produce complex distributions of data, which can be used to represent the full range of valid, diverse actions in continuous action spaces rather than collapsing them into a single averaged answer (Chi et al., 2023).

A Diffusion Model has two main components: a forward process and a reverse process. The forward process progressively adds gaussian noise to a sample of data until the sample becomes completely random noise. The reverse process is a neural network that learns to predict and remove the noise added at each step with a concrete training objective: predict the noise added at a random step. Instead of learning to generate a sample from scratch, the network learns to denoise a noisy sample. By replicating this process, the trained network can generate new samples from pure noise.

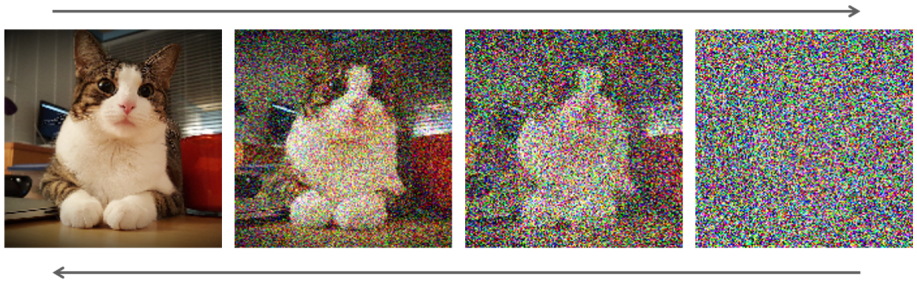


Figure 1. A diffusion model progressively adds noise to a sample in the forward process, then learns to reverse this process step by step to recover a clean sample from pure noise (Vahdat and Kreis, 2022).

2.1 Forward process

The forward process takes a real data sample x_0 and progressively adds a small amount of Gaussian noise to it over T steps, until x_T is indistinguishable from pure random noise, as illustrated in Figure 1. At each step, the previous sample is scaled down slightly and a small amount of noise is added:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}\right), \tag{2}$$

where β_t is a small, fixed noise variance that increases with each step t, and $\mathcal{N}$ denotes a Gaussian distribution (Ho et al., 2020). Crucially, this forward process is fixed rather than learned: β_t is chosen ahead of time according to a schedule, and no network parameters are involved in going from x_0 to x_T. It exists only to generate training pairs of a clean sample and a corresponding noisy version, which the reverse process is then trained to undo.

2.2 Reverse process

The reverse process is the part of a diffusion model that is learned. Since the exact reverse of the forward process, $q(x_{t-1} \mid x_t)$, cannot be computed directly, a neural network with parameters θ is trained to approximate it:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1};\ \mu_\theta(x_t, t),\ \Sigma_\theta(x_t, t)), \tag{3}$$

where the network predicts the mean μ_θ and variance Σ_θ of the slightly-less-noisy sample x_{t-1}, given the current noisy sample x_t and the current step t. Ho et al. (2020) show that this reduces to a much simpler solution: rather than predicting μ_θ and Σ_θ directly, the network is instead trained to predict the noise ϵ that was added to x_0 to produce x_t, by minimising the mean squared error between the predicted and actual noise:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, \epsilon, t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right], \tag{4}$$

where $\epsilon_\theta(x_t, t)$ is the network's predicted noise. At generation time, the network starts from random noise x_T and repeatedly applies its learned noise prediction to remove a small amount of noise at each step, until a clean sample x_0 remains, as illustrated by the right-to-left direction in Figure 1.

2.3 From images to actions

Applying diffusion concepts to reinforcement learning requires changing both what is "diffused" and how it is generated. Firstly, the data sample x_0 is no longer a grid of pixel values, but a vector representing a continuous action, or a short sequence of future actions (Chi et al., 2023). Secondly, the network needs to know what state it is acting in, so the current state s is fed into the network at every denoising step, alongside the noisy action. This turns the network into a policy $\pi_\theta(a \mid s)$: starting from random noise, it is repeatedly denoised, using the state as context each time, until a usable action remains. Chi et al. (2023) predict a short sequence of future actions at once for smoother behaviour. This paper adopts short action chunks because landing requires coordinated control over multiple timesteps.

3 Related works

3.1 Discretised and continuous-action value-based control

Deep Q-Networks (DQN) combine Q-learning with a neural network over a discrete set of actions (Mnih et al., 2015). Since LunarLanderContinuous-v3 has two continuous action dimensions, both must first be discretised before DQN can be used. Splitting each dimension into K bins gives K^2 joint actions. Double DQN separates action selection from evaluation to improve value estimates (van Hasselt et al., 2015), yet it uses the same discretised action space. DDPG learns a deterministic policy directly in continuous action space (Lillicrap et al., 2015). DDPG still produces one action for a given state, while a discretised DQN selects one action bin. A diffusion policy can learn a distribution over valid controls. None of this settles whether that distributional capacity actually helps once the training data itself has little multimodal behaviour to represent: DQN, Double DQN, and DDPG are all compared under demonstrations or rewards that were never set up to test that question directly.

3.2 Behaviour cloning and continuous-control demonstrations

In behaviour cloning (BC), a policy is trained by supervised learning on state-action pairs collected from an expert. The BC and diffusion students in this study receive the same SAC demonstrations and predict the same type of action chunk. This makes BC the direct baseline. Because the BC model is deterministic, it returns one chunk for each state and may average together different valid behaviours. It is trained only on states found in the demonstrations. An early prediction error may take the policy outside that distribution. Later errors may then build on it (Ross et al., 2011; Pearce et al., 2023).

Soft Actor-Critic (SAC) uses an off-policy, maximum-entropy actor-critic objective for continuous actions (Haarnoja et al., 2018). Its policy is stochastic, so it can collect varied successful trajectories. Here, SAC generates the demonstrations used to train both students. The imitation-learning comparison is between BC and diffusion. Neither line of work says much about what happens once the expert generating the demonstrations settles into a near-deterministic strategy, which is what a trained SAC policy tends to do once there's little entropy left worth trading for reward. The compounding-error argument against BC assumes demonstrations rich enough to expose distribution shift in the first place, and that's the assumption this paper's setup puts under strain.

3.3 Diffusion models for action generation

Janner et al. (2022) framed planning as the denoising of a full state-action trajectory. The method can represent multimodal, long-horizon behaviour. Generating a new trajectory whenever the state changes, however, is expensive for online control. Pearce et al. (2023) used diffusion directly for behaviour cloning. Chi et al. (2023) combined DDPM-style denoising (Ho et al., 2020) with state conditioning and receding-horizon action chunks. The diffusion student in this study follows that setup in LunarLanderContinuous-v3. Its comparison with BC tests diffusion-based action generation against a deterministic model trained on the same demonstrations. The discretised DQN provides a separate value-based baseline. None of these studies hold the demonstration source fixed and ask whether it's multimodality in that source, rather than the action space simply being continuous or multidimensional, that diffusion's advantage actually depends on. That's the gap this paper's comparison is built to test.

Table 1. Summary of related research and its relevance to this study

Study	Contribution	Relevance
Mnih et al. (2015)	DQN for discrete control	Its ceiling comes from the discretisation resolution, not from what it's able to represent
Ross et al. (2011)	Distribution shift in imitation learning	Assumes demonstrations multimodal enough for shift to actually bite; this paper's expert is closer to deterministic
Haarnoja et al. (2018)	SAC for stochastic continuous control	Generates demonstrations whose multimodality shrinks as training converges, which is the condition tested here
Ho et al. (2020)	DDPM: iterative denoising generative model	Supplies the denoising mechanism, not a claim about when that capacity is actually worth using
Janner et al. (2022)	Trajectory-level diffusion for planning	Shows diffusion can capture multimodal behaviour when it exists, not what happens when it doesn't
Pearce et al. (2023)	Diffusion-based behaviour cloning	Compares diffusion with BC, but not with demonstrations deliberately low in multimodality
Chi et al. (2023)	State-conditioned action diffusion	Basis for the diffusion student; the reason to test its edge against a near-unimodal expert

4 Experiments

4.1 Environment

This study uses LunarLanderContinuous-v3 from Gymnasium (Towers et al., 2024) rather than a single-joint benchmark such as Pendulum-v1, because its action space is two-dimensional instead of one-dimensional: the main engine and the two side engines must be commanded together, and a successful landing depends on coordinating both at once. A diffusion policy's advantage over simpler imitation baselines is expected to grow with action-space dimensionality, and a one-dimensional control problem gives a diffusion model little room to show any improvement over a deterministic baseline.

The environment gives an eight-dimensional observation (horizontal and vertical position, horizontal and vertical velocity, angle, angular velocity, and two leg-contact flags) and a two-dimensional continuous action, each value bounded in $[-1, 1]$, for the main and side engines respectively.

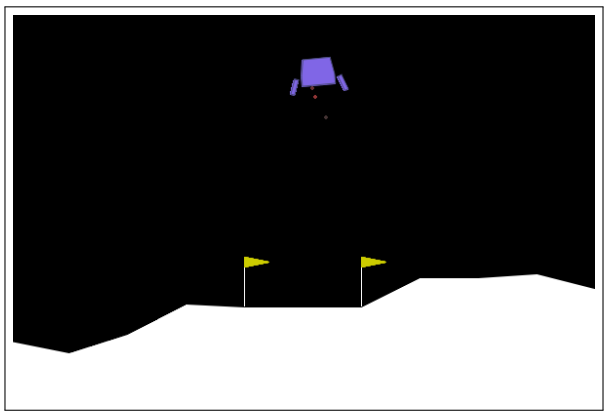


Figure 2. A mid-flight frame from LunarLanderContinuous-v3. The lander must fire the main and side engines together to descend between the two flags marking the landing pad.

4.2 Preprocessing

States are normalised with the training split's mean and standard deviation, reused unchanged at validation and evaluation time. Each state is paired with a chunk of the next eight expert actions rather than one action, since both students predict short sequences. The DQN baseline discretises each of the two action dimensions into five bins, giving 25 joint actions.

4.3 Baseline model

The baseline is a discretised DQN (Mnih et al., 2015) Q-network with two 256-unit ReLU hidden layers that maps the 8-dimensional state to one Q-value per discrete action, and a target network of the same architecture that is synchronised from the online network every 1,000 steps. Actions are selected with an epsilon-greedy policy, decaying linearly from 1.0 to 0.05 over 150,000 steps, and transitions are stored in a replay buffer of 200,000 entries sampled in minibatches of 256, both standard components of the original DQN algorithm (Mnih et al., 2015). Training minimises the Huber (smooth L1) loss between predicted Q-values and TD targets, $r_t + \gamma \max_a Q_{\text{target}}(s_{t+1}, a)$ with $\gamma = 0.99$, using the Adam optimiser (Kingma and Ba, 2014) at a learning rate of 3×10^{-4}, and training stops early once greedy evaluation return exceeds 200 for three consecutive checks.

4.4 Advanced model

The Diffusion Policy (Chi et al., 2023) replaces the baseline's single predicted output with a small denoising network that is conditioned on the state. Instead of outputting one Q-value per discrete action or one fixed action chunk, the network takes a noisy 8-step action chunk a_t, the current noise level (timestep) t, and the normalised state s, and predicts the noise $\epsilon_\theta(a_t, t, s)$ that would need to be removed to recover the clean chunk, following the noise-prediction parameterisation of DDPM (Ho et al., 2020). The noisy chunk, a sinusoidal embedding of t, and the state are concatenated and passed through two 512-unit hidden layers with Mish activations, so the same network learns to denoise at every noise level rather than needing a separate model per level.

The forward process (Ho et al., 2020) is applied to action chunks during training. A clean chunk a_0 from the demonstration data is mixed with Gaussian noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ at a randomly sampled timestep t,

$$a_t = \sqrt{\bar{\alpha}_t}\, a_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \tag{5}$$

where $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t$ is the cumulative product of α_t up to step t. The network is trained to recover ϵ from a_t, t, and s by minimising

$$\mathcal{L}(\theta) = \mathbb{E}_{a_0, \epsilon, t, s}\left[\|\epsilon - \epsilon_\theta(a_t, t, s)\|^2\right], \tag{6}$$

which is the same noise-prediction loss as Equation 4, now conditioned on the state so the denoised chunk matches what the demonstrator actually did from that state.

At inference time there is no clean chunk to noise, only the current state. Sampling starts from pure noise $a_T \sim \mathcal{N}(0, \mathbf{I})$ and repeats, for $t = T, \ldots, 1$,

$$a_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(a_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\, \epsilon_\theta(a_t, t, s)\right) + \sqrt{\beta_t}\, z, \qquad z \sim \mathcal{N}(0, \mathbf{I}), \tag{7}$$

over $T = 30$ steps. Only the first four of the eight predicted actions are executed before the controller observes the new state and samples a fresh chunk, following the receding-horizon action-chunking scheme of Chi et al. (2023), so the diffusion policy replans as often as the DQN baseline even though each chunk takes 30 denoising passes to produce.

4.5 Comparison models

Two further controllers give the discretised DQN and the diffusion policy something to compare against. The SAC expert (Haarnoja et al., 2018) is the teacher, not a student: it trains directly on the environment reward with Soft Actor-Critic, a continuous-action actor-critic method that maximises a return objective augmented with a policy-entropy bonus,

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \alpha\, \mathcal{H}(\pi(\cdot \mid s_t))\right)\right], \tag{8}$$

where $\mathcal{H}(\pi(\cdot \mid s_t))$ is the entropy of the action distribution at state s_t and α controls how much that entropy is worth relative to reward. Maximising entropy keeps the policy exploring stochastically

during training, but once training converges on a near-optimal landing strategy there is little reward left to trade for extra randomness, so the trained policy's action distribution narrows and its rollouts become close to deterministic. This is why the demonstrations it generates for the students to imitate carry limited multimodality. Its rollouts become the demonstration data both students imitate, and its own results are the highest imitation can reach, since neither student ever sees the reward function, but only the expert's actions.

The MLP student isolates what the diffusion policy actually adds over plain behaviour cloning. It is a feedforward network with two 256-unit ReLU hidden layers that maps the normalised state directly to an 8-step action chunk, but instead of learning to denoise it regresses straight to the demonstrator's actions by minimising

$$\mathcal{L}(\theta) = \mathbb{E}_{a_0,s}\left[\|a_0 - f_\theta(s)\|^2\right], \tag{9}$$

where $f_\theta(s)$ is the MLP's predicted action chunk for state s and a_0 is the demonstrator's actual chunk.The MLP regresses directly to a single target action, while the diffusion student regresses to the noise needed to recover a sample from a distribution over plausible actions.

4.6 Experimental setup and evaluation

Table 2. Experimental configuration used for the controlled comparison.

Setting	Value
Framework	PyTorch, with the SAC expert from Stable-Baselines3
Random seeds	42 (train); 30 fixed evaluation seeds (50,000–50,029)
Batch size	256 for the DQN, MLP, and diffusion student
Learning rate	3×10^{-4} (Adam) for all three trained networks
Optimiser	Adam (Kingma and Ba, 2014)
Epochs or episodes	DQN: up to 1,200 episodes (early-stopped at 450); SAC: 300,000 timesteps; MLP/diffusion students: 20,000 gradient steps each
Model-selection rule	DQN/SAC: checkpoint with the best periodic greedy-evaluation return; students: checkpoint with the lowest validation loss

All four controllers, discretised DQN, SAC expert, MLP student, diffusion student, are evaluated on the same 30 fixed seeds. We score each on four metrics: episodic return, landing-success rate (episodes ending with a positive final reward rather than a crash or timeout), action smoothness (mean change between consecutive action vectors, lower is steadier), and per-action inference time. Return and success rate answer whether the diffusion policy imitates the expert better than the baselines. Smoothness and inference time catch the practical costs a return-only comparison would miss. Rollouts are stochastic, so each metric is a mean, with standard deviation for return, over the 30 evaluation episodes.

5 Discussion

MLP behaviour cloning ties the SAC expert for the best landing-success rate and produces the smoothest, fastest actions, though its mean return is not clearly higher than the DQN's or the SAC expert's once the spread across seeds is taken into account. The diffusion policy stayed close to the discretised DQN without beating it, and neither student fully matched the SAC expert they were trained to imitate (Table 3).

Table 3. Evaluation results over 30 matched seeds (mean ± std for return).

Controller	Return	Success rate	Smoothness	Inference time
MLP student	231.0 ± 54.4	96.7%	0.1171	0.083 ms
Diffusion student	228.5 ± 78.6	90.0%	0.5676	3.766 ms
Discretised DQN	233.4 ± 86.5	86.7%	0.6819	0.306 ms
SAC expert	233.2 ± 46.3	96.7%	0.1829	0.737 ms

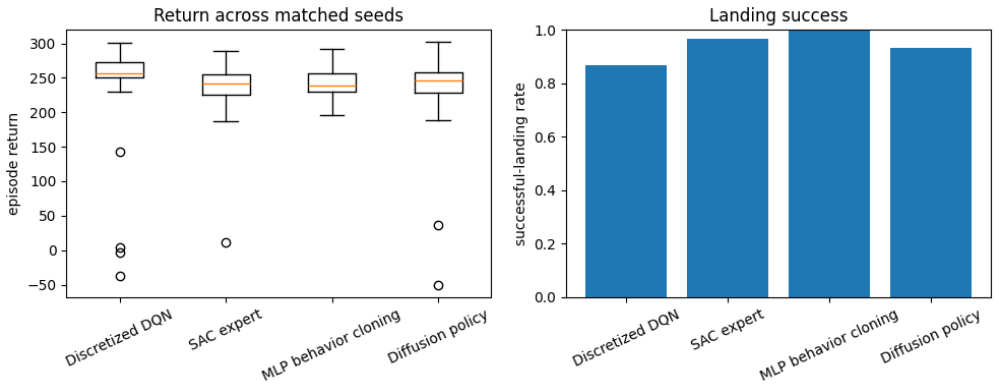


Figure 3. Return distribution and landing-success rate for each controller across the 30 matched evaluation seeds.

The MLP student's showing only partly agrees with Ross et al. (2011) and Pearce et al. (2023): distribution shift is the usual failure mode for deterministic behavior cloning, yet the MLP student here matched the SAC expert's landing-success rate (96.7%). This is probably because the SAC expert (Haarnoja et al., 2018) is close to deterministic once it has learned a near-optimal landing strategy, leaving little of the multimodal behaviour that Chi et al. (2023) and Janner et al. (2022) say diffusion policies handle better than single-point regression. With no genuinely multimodal demonstrations to model, the diffusion policy's distributional capacity goes unused: it still has to find the same near-unique action the MLP regresses to directly, just through 30 denoising passes per chunk instead of one forward pass. That matches its much higher inference time (3.766 ms vs. 0.083 ms) and lower smoothness (0.5676 vs. 0.1171) in Table 3. The discretised DQN (Mnih et al., 2015) trailing both students on success rate fits its own baseline description in Section 3: binning the two action dimensions into 25 joint actions caps control precision below what either continuous-output student can represent, which likely explains its lower success rate (86.7%) despite a comparable, and even marginally higher, mean return. Therefore, the results back up prior work on DQN's discretisation cost, but don't confirm diffusion's expected edge over behavior cloning because that edge depends on multimodal demonstration data that the SAC expert did not provide.

A separate one-dimensional comparison, submitted alongside this paper as `paper_pendulum.ipynb`, gives this some additional evidence. It runs the same diffusion-versus-value-based setup on Pendulum-v1, where a single torque dimension leaves little room for coordinated, multi-axis control. There, a discretised Double-DQN teacher reaches a mean return of -137.2 ± 65.3, while a diffusion policy behaviour-cloned from that teacher's rollouts manages only -1017.6 ± 171.0: a far bigger gap than the near-parity we saw between the DQN and diffusion policy on LunarLanderContinuous-v3. This isn't a controlled ablation of dimensionality alone, since both notebooks use different teachers, network sizes, and training budgets. Nevertheless, the direction matches the paper's motivating hypothesis: diffusion's standing against a discretised value-based baseline was far worse on the one-dimensional task than on the two-dimensional one.

5.1 Limitations

We trained the SAC expert, the DQN, and the MLP student on a single seed each, so their individual results may not hold for a different seed, expert, or run. The diffusion student is the one exception: since it's the model the paper's hypothesis depends on most, it was retrained twice more from scratch with independent seeds (123 and 2024), holding the demonstration data and evaluation seeds fixed. Its return came out to 215.9 ± 94.0 and 248.7 ± 53.5 for those two seeds, against 228.5 ± 78.6 for the original run, giving a mean of 231.0 ± 13.5 across the three training seeds. That spread across seeds (13.5) is well inside the spread across evaluation episodes within any single run (54–94), so training-seed variance doesn't look like the main source of noise in the diffusion student's numbers specifically, though this check doesn't extend to the DQN, SAC, or MLP, which remain single-seed. Hyperparameters were also fixed rather than tuned per model: the MLP, diffusion, and DQN networks all share the same learning rate, batch size, and optimiser, so part of the gap between them could come from that shared configuration rather than each model's best possible setup. The comparison also covers only LunarLanderContinuous-v3, so the paper's core hypothesis, that diffusion helps more as action dimensionality grows, is not tested directly against a lower- or higher-dimensional environment. The SAC expert itself is close to deterministic once trained, which limits how multimodal the demonstrations are and, in turn, how much room the diffusion policy has to show any advantage. Finally, action smoothness and inference time are useful proxies for deployability, but they do not

capture how severe a failure is or how close a landing came to crashing.

6 Conclusions

This study asked whether a diffusion policy can imitate continuous expert behaviour in LunarLanderContinuous-v3 better than standard behaviour cloning and a discretised DQN baseline. It did not: the discretised DQN, which bins each of the two action dimensions into five values for 25 joint actions, reached a mean return of 233.4 ± 86.5 and an 86.7% success rate, while the diffusion policy, which denoises an 8-step action chunk over 30 reverse steps and executes four actions before replanning, reached 228.5 ± 78.6 return and 90.0% success. The MLP student, trained on the same data and action chunks but regressing to the demonstrator's actions directly, matched the SAC expert's landing-success rate (96.7%) while giving the lowest action smoothness and the fastest inference, at 231.0 ± 54.4 return. The most likely reason is that the SAC expert's demonstrations were close to deterministic, leaving little multimodal behaviour for a diffusion policy to model in the first place. Retraining the diffusion student on two further seeds (215.9 ± 94.0 and 248.7 ± 53.5, against 228.5 ± 78.6 for the reported run) suggests this outcome isn't just one unlucky training run, at least for that model. The main takeaway is that a diffusion policy's advantage over plain behaviour cloning depends on the demonstrations actually containing several valid ways to act from the same state, not just on the action space being continuous or multidimensional.

References

Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R. and Song, S. (2023), 'Diffusion policy: Visuomotor policy learning via action diffusion', *arXiv preprint arXiv:2303.04137* .
URL: *https://arxiv.org/pdf/2303.04137*

Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018), Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, *in* 'Proceedings of the 35th International Conference on Machine Learning', Vol. 80, PMLR, pp. 1861–1870.
URL: *https://proceedings.mlr.press/v80/haarnoja18b.html*

Ho, J., Jain, A. and Abbeel, P. (2020), 'Denoising diffusion probabilistic models', *arXiv preprint arXiv:2006.11239* .
URL: *https://arxiv.org/pdf/2006.11239*

Janner, M., Du, Y., Tenenbaum, J. B. and Levine, S. (2022), 'Planning with diffusion for flexible behavior synthesis', *arXiv preprint arXiv:2205.09991* .
URL: *https://arxiv.org/abs/2205.09991*

Kingma, D. P. and Ba, J. (2014), 'Adam: A method for stochastic optimization', *arXiv preprint arXiv:1412.6980* .
URL: *https://arxiv.org/abs/1412.6980*

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. and Wierstra, D. (2015), 'Continuous control with deep reinforcement learning', *arXiv preprint arXiv:1509.02971* .
URL: *https://arxiv.org/abs/1509.02971*

Milvus (n.d.), 'How does RL apply to continuous control problems?'. Accessed: 2026-07-17.
URL: *https://blog.milvus.io/ai-quick-reference/how-does-rl-apply-to-continuous-control-problems*

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.
URL: *https://www.nature.com/articles/nature14236*

Pearce, T., Rashid, T., Kanervisto, A., Bignell, D., Sun, M., Georgescu, R., Macua, S. V., Tan, S. Z., Momennejad, I., Hofmann, K. and Devlin, S. (2023), 'Imitating human behaviour with diffusion models', *arXiv preprint arXiv:2301.10677* .
URL: *https://arxiv.org/abs/2301.10677*

Ross, S., Gordon, G. J. and Bagnell, J. A. (2011), A reduction of imitation learning and structured prediction to no-regret online learning, *in* 'Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics', Vol. 15, PMLR, pp. 627–635.
URL: *https://proceedings.mlr.press/v15/ross11a.html*

Terven, J. (2025), 'Deep reinforcement learning: A chronological overview and methods', *AI* **6**(3), 46.
URL: *https://www.mdpi.com/2673-2688/6/3/46*

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H. and Younis, O. G. (2024), 'Gymnasium: A standard interface for reinforcement learning environments', *arXiv preprint arXiv:2407.17032* .
URL: *https://arxiv.org/abs/2407.17032*

Vahdat, A. and Kreis, K. (2022), 'Improving diffusion models as an alternative to GANs, part 2', NVIDIA Technical Blog. Accessed: 2026-07-20.
URL: *https://developer.nvidia.com/blog/improving-diffusion-models-as-an-alternative-to-gans-part-2/*

van Hasselt, H., Guez, A. and Silver, D. (2015), 'Deep reinforcement learning with double Q-learning', *arXiv preprint arXiv:1509.06461* .
URL: *https://arxiv.org/abs/1509.06461*

A Additional experimental details

The demonstration dataset comes from 200 SAC expert episodes, collected with reset seeds 10,000 to 10,199. These are split 80/20 by whole episode, not by individual transition, so no state from a training episode also turns up in validation: 160 episodes for training, 40 for validation, giving 61,250 training transitions and 16,029 validation transitions. State normalisation statistics (mean and standard deviation) come from the training split only and are then applied to validation and evaluation states, so no validation information leaks into training. Figure 4 shows the spread of returns across those episodes and the resulting distribution of first-action values that both students are trained to reproduce.

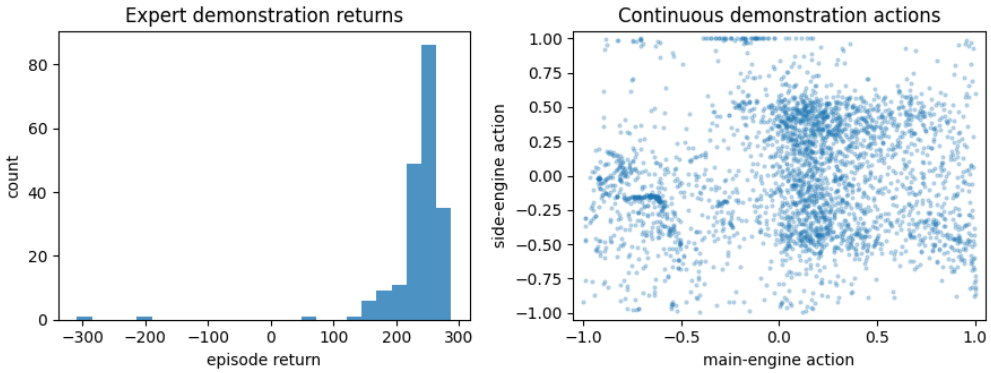


Figure 4. Left: distribution of episodic return across the 200 SAC expert demonstration episodes. Right: distribution of the expert's continuous action values sampled from those demonstrations.

Both students train for 20,000 gradient steps and checkpoint at whichever step has the lowest validation loss. The MLP student's validation loss, mean squared error between its predicted action chunk and the demonstrator's, falls from 0.0729 at step 500 to 0.0552 at step 20,000. The diffusion student's validation loss, mean squared error between predicted and true noise, falls from 0.4729 to 0.3365 over the same steps. These two losses sit on different scales, since one regresses actions directly and the other regresses noise, so they should not be compared to each other directly.

The DQN's gradient norm is also clipped to 10.0 before each optimiser step, on top of the target-network synchronisation and epsilon-greedy schedule already described in Section 4.3. Figure 5 shows how its training actually went: raw per-episode return, a 25-episode moving average, and the periodic greedy-evaluation checkpoints, up to the early-stopping point from that same section.

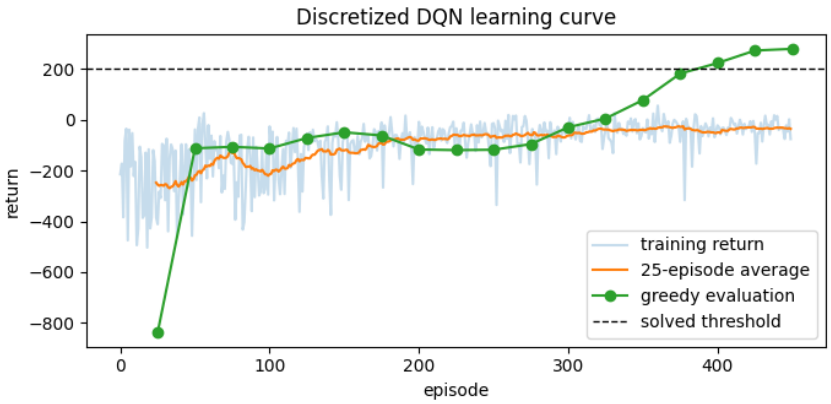


Figure 5. Discretised DQN learning curve: raw training return, 25-episode moving average, and periodic greedy-evaluation return.